

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

федеральное государственное автономное образовательное
учреждение высшего образования



**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Инженерная школа информационных технологий и робототехники

09.04.04 «Программная инженерия»

Отделение информационных технологий

Отчет по лабораторной работе №5

по дисциплине «Управление разработкой промышленного программного обеспечения»

Выполнил:

Студент группы 8МП5Л

_____ Шаврин А.П.

Проверил:

К.т.н., доцент ОИТ

_____ Небаба С.Г.

Задание

Задание: подключить автопроверку Python-скрипта на соответствие соглашению PEP-8

1. Познакомиться с соглашением PEP-8
2. Реализовать код на Python, решающий небольшую задачку (например, парсер или конвертер). Минимум 30 строк кода
3. Установить инструмент `pycodestyle` и провести проверку с помощью данного инструмента вручную (через CLI). Какие ошибки были обнаружены? Ошибки исправлять не нужно.
4. Сгенерировать с помощью `pip freeze` файл `requirements.txt`
5. Подключить GitHub Actions к проекту в GitHub. В файл `.github/workflows/check.yml` прописать следующие сценарии:
 - Установка необходимых файлов из `requirements.txt`
 - Предварительная установка `pycodestyle` в конфигурационном файле
 - Реализовать внутренний скрипт для проверки на наличие ошибок. Если ошибки есть - скрипт завершается с ошибкой. Подсказка для продвинутых: некоторые ошибки можно исключать при должном обосновании с помощью передачи дополнительного специального параметра.
6. Загрузить написанный python-скрипт. Внимание: для первого push'a с .ру-файлом важно, чтобы была хотя бы одна ошибка. Workflow check должен завершиться со статусом `failure` (это также должно быть отображено в GitHub, в истории коммитов в виде маленьких меток у каждого коммита, и в Actions). Необходимо предоставить детализацию, почему именно возвращена ошибка (для проверки, правильно ли реализовано задание 4).
7. Внести исправления в .ру-файл. Не должно остаться ни одной ошибки от `pycodestyle`. Снова сделать push. Workflow check должен быть завершен со статусом `success`: это также необходимо продемонстрировать.

Ход работы

Перед выполнением работы было изучено руководство по стилю кода Python PEP-8.

Создан файл src/converter.py, который выполняет преобразование CSV-файла в JSON. Код содержит намеренные нарушения PEP-8 (отсутствие пробелов после запятых, вокруг операторов, неправильное количество пустых строк и т.д.). (исходный код представлен на рисунке 1)

```
1  import json
2  import csv
3  import os
4
5  def read_csv(file_path):
6      """Читает CSV файл и возвращает список строк."""
7      with open(file_path, 'r') as f:
8          reader = csv.reader(f)
9          data=[row for row in reader]
10         return data
11
12     def convert_to_json(data,output_file):
13         """Конвертирует данные из CSV в JSON."""
14         # Преобразуем список строк в список словарей (первая строка - заголовки)
15         headers=data[0]
16         json_data=[]
17
18         for row in data[1:]:
19             item={}
20             for i,value in enumerate(row):
21                 item[headers[i]]=value
22             json_data.append(item)
23
24         with open(output_file,'w') as f:
25             json.dump(json_data,f,indent=2)
26
27         return True
28
29     if __name__=="__main__":
30         input_csv="data.csv"
31         output_json="data.json"
32
33         # Проверка существования входного файла
34         if not os.path.exists(input_csv):
35             print(f"Ошибка: файл {input_csv} не найден!")
36             exit(1)
37
38         data=read_csv(input_csv)
39         convert_to_json(data,output_json)
40
41         # Вывод количества записей (кроме заголовка)
42         print(f"Конвертировано записей: {len(data)-1}")
43         print(["Готово!"])
```

Рисунок 1. Исходный код

Для проверки стиля был установлен инструмент rucodestyle. Командой rucodestyle src/converter.py была запущена проверка файла в ходе которой были обнаружены ошибки представленные на рисунке 2.

```
~/Doc/D/LetiMaster/2_semester/IndustrialSoftwareDevelopmentManagement/lab5 | main .....
pycodestyle src/converter.py
src/converter.py:5:1: E302 expected 2 blank lines, found 1
src/converter.py:9:13: E225 missing whitespace around operator
src/converter.py:12:1: E302 expected 2 blank lines, found 1
src/converter.py:12:25: E231 missing whitespace after ','
src/converter.py:15:12: E225 missing whitespace around operator
src/converter.py:16:14: E225 missing whitespace around operator
src/converter.py:19:13: E225 missing whitespace around operator
src/converter.py:20:14: E231 missing whitespace after ','
src/converter.py:21:29: E225 missing whitespace around operator
src/converter.py:24:26: E231 missing whitespace after ','
src/converter.py:25:28: E231 missing whitespace after ','
src/converter.py:25:30: E231 missing whitespace after ','
src/converter.py:29:1: E305 expected 2 blank lines after class or function definition, found 1
src/converter.py:29:12: E225 missing whitespace around operator
src/converter.py:30:14: E225 missing whitespace around operator
src/converter.py:31:16: E225 missing whitespace around operator
src/converter.py:32:1: W293 blank line contains whitespace
src/converter.py:37:1: W293 blank line contains whitespace
src/converter.py:38:9: E225 missing whitespace around operator
src/converter.py:39:25: E231 missing whitespace after ','
src/converter.py:40:1: W293 blank line contains whitespace
src/converter.py:44:1: W293 blank line contains whitespace
src/converter.py:44:5: W292 no newline at end of file
```

Рисунок 2. Результат проверки pycodestyle.

Файл зависимостей сгенерирован командой: `uv pip freeze > requirements.txt`. Содержимое `requirements.txt` включало только `pycodestyle==2.11.1` (Остальные пакеты отсутствуют.).

Затем в репозитории создана папка `.github/workflows` и файл `check.yml` со следующим содержимым представленным на рисунке 3. В этом файле сначала устанавливаются зависимости из `requirements.txt`, затем явно устанавливается `pycodestyle`, после чего выполняется проверка всех `.py`-файлов в репозитории. Параметр `--max-line-length=99` позволяет не учитывать превышение 79 символов (использован для демонстрации настройки).

```

1  name: lab5(PEP-8 Check)
2
3  on: [push, pull_request]
4
5  jobs:
6    lint:
7      runs-on: ubuntu-latest
8
9      steps:
10     - name: Checkout code
11       uses: actions/checkout@v4
12
13     - name: Set up Python
14       uses: actions/setup-python@v5
15       with:
16         python-version: '3.12'
17
18     - name: Install dependencies
19       run: |
20         python -m pip install --upgrade pip
21         if [ -f requirements.txt ]; then pip install -r requirements.txt; fi
22
23     - name: install pycodestyle
24       run: pip install pycodestyle
25
26     - name: Run pycodestyle
27       run: |
28         pycodestyle --max-line-length=99 **/*.py
29

```

Рисунок 3. check.yml

Все файлы были добавлены в индекс и закоммичены. После push'a GitHub Actions автоматически запустил workflow. На рисунке 4 видно, что запуск завершился с красным значком (failure).

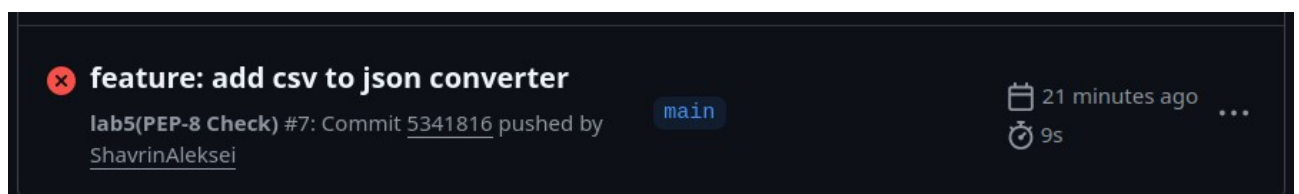


Рисунок 4. Failed action.

В логах на рисунке 5 отображаются все найденные pycodestyle ошибки.

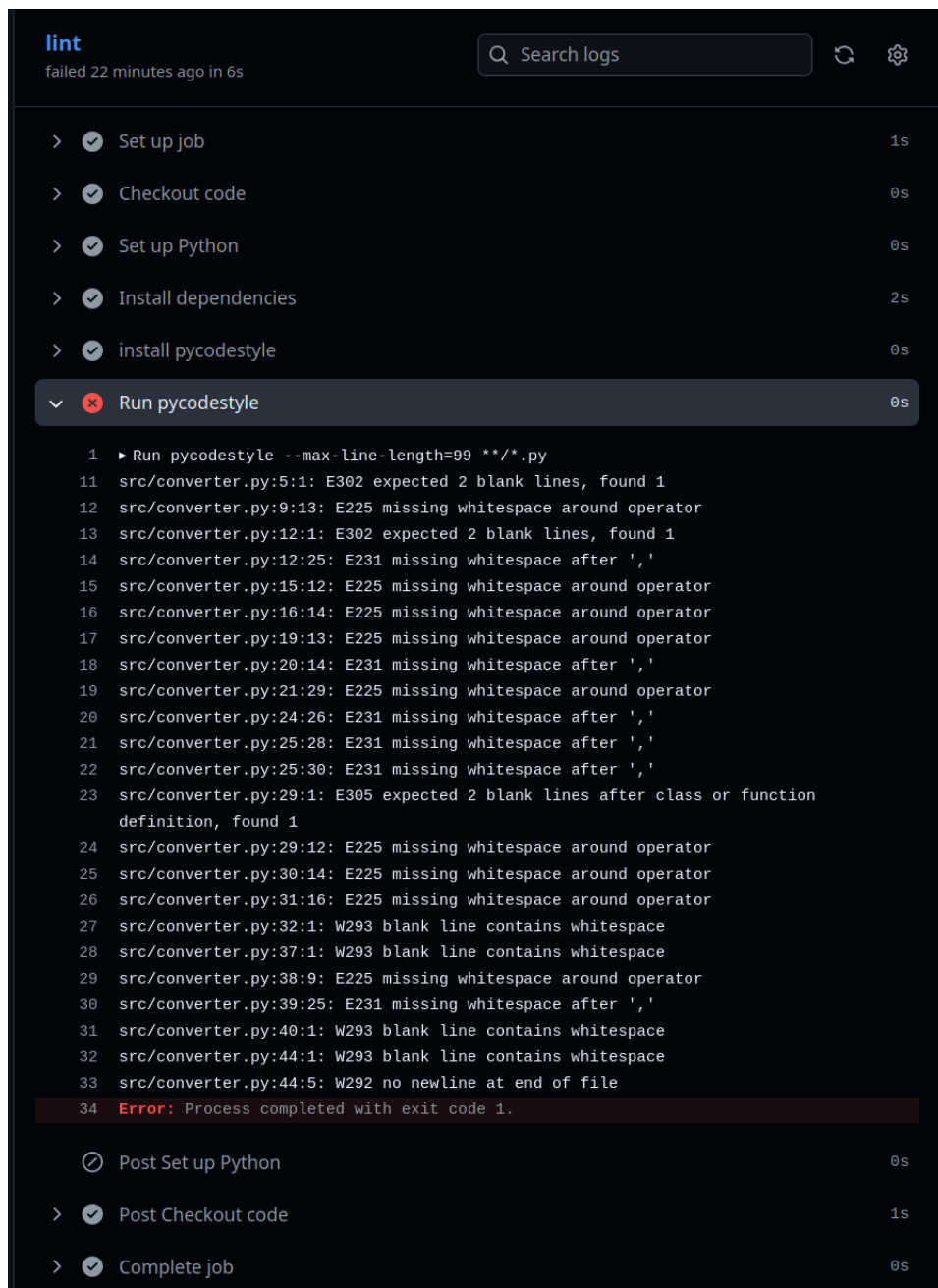


Рисунок 5. Лог ошибок акшена.

После все недочеты были исправлены вручную итоговый файл представденн на рисунке 6. Локальная проверка `pycodestyle` не показала ошибок (рисунок 7). После чего был выполнен `push` исправлений, в результате которого запустился новый action, который так же благополучно прошел проверки (рисунок 8).

```

1  import json
2  import csv
3  import os
4
5
6  def read_csv(file_path):
7      """Читает CSV файл и возвращает список строк."""
8      with open(file_path, 'r') as f:
9          reader = csv.reader(f)
10         data = [row for row in reader]
11     return data
12
13
14  def convert_to_json(data, output_file):
15      """Конвертирует данные из CSV в JSON."""
16      # Преобразуем список строк в список словарей (первая строка - заголовки)
17      headers = data[0]
18      json_data = []
19
20      for row in data[1:]:
21          item = {}
22          for i, value in enumerate(row):
23              item[headers[i]] = value
24          json_data.append(item)
25
26      with open(output_file, 'w') as f:
27          json.dump(json_data, f, indent=2)
28
29      return True
30
31
32  if __name__ == "__main__":
33      input_csv = "data.csv"
34      output_json = "data.json"
35
36      # Проверка существования входного файла
37      if not os.path.exists(input_csv):
38          print(f"Ошибка: файл {input_csv} не найден!")
39          exit(1)
40
41      data = read_csv(input_csv)
42      convert_to_json(data, output_json)
43
44      # Вывод количества записей (кроме заголовка)
45      print(f"Конвертировано записей: {len(data)-1}")
46      print("Готово!")
47

```

Рисунок 6. Исходный код исправленный

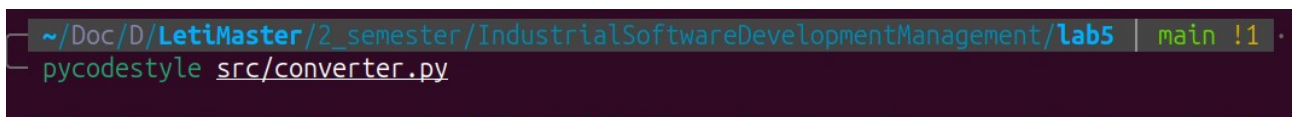


Рисунок 7. Результат проверки pycodestyle локально.

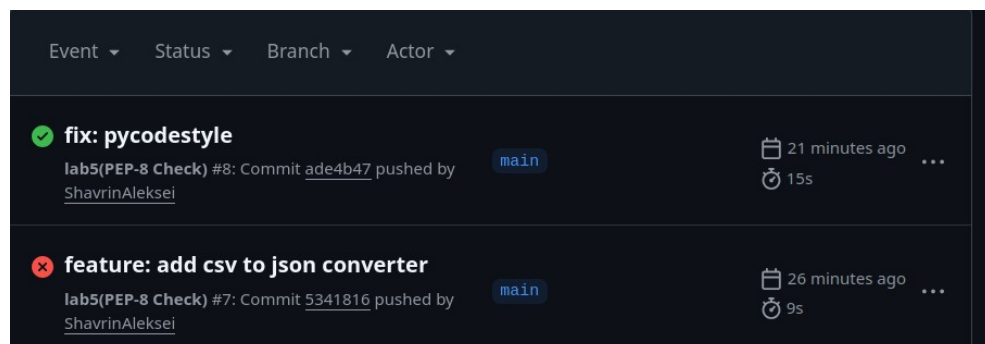


Рисунок 8. Success action.

В логах шаг Run pycodestyle не выводит ошибок и завершается с кодом 0. Все шаги выполнены успешно.

> **Annotations**
1 warning

lint

succeeded 22 minutes ago in 11s

Q Search logs

↺ ⚙

> ✓ Set up job1s

> ✓ Checkout code1s

> ✓ Set up Python0s

> ✓ Install dependencies4s

> ✓ install pycodestyle0s

> ✓ Run pycodestyle0s

1 ▶ Run pycodestyle --max-line-length=99 **/*.py

> ✓ Post Set up Python0s

> ✓ Post Checkout code1s

> ✓ Complete job0s

Рисунок 9. Лог успешного акшена.

Вывод

В ходе выполнения задания была успешно настроена автоматическая проверка стиля кода Python на соответствие PEP-8 с использованием GitHub Actions. Был написан скрипт `converter.py` (более 30 строк) с намеренными нарушениями PEP-8, которые были выявлены при локальном запуске `pycodestyle`. Сгенерирован файл `requirements.txt`, содержащий `pycodestyle`. В репозитории создан файл `.github/workflows/check.yml`, описывающий workflow: установка зависимостей, установка `pycodestyle` и запуск проверки всех `.py`-файлов.

При первом push'e workflow завершился с ошибкой (failure), поскольку `pycodestyle` обнаружил нарушения PEP-8 – это соответствует требованию задания. После исправления всех ошибок и повторного push'a workflow выполнен успешно (success).